DV Track

# Lab T-DV2: UVM Environment for the Cache

Agents, Sequences, Factory, Configuration and Phases

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

**Lab objective:** Rebuild the Lab T-DV1 testbench of the data cache with the Universal Verification Methodology (UVM): a reusable `mem_if` agent used three times, sequences and a virtual sequence, factory overrides, configuration through `uvm_config_db`, and a scoreboard that reuses the Lab T-MOD cache model unchanged.

| | |
|---|---|
| **Duration** | 3 weeks (approximately 45–60 hours) |
| **Prerequisites** | Labs T-MOD and T-DV1 (and through them the common core, in particular R3, R4, V2 and V3) |
| **Designs** | `dcache` with `arb_mem` (unchanged); new UVM agent `mem_if_agent` and environment `hw/ip/dcache/dv_uvm/` |
| **Reused in** | Last lab of the DV track; the `mem_if` agent and the `SIM` flow are reused by every later UVM environment |

## Contents


**1 Overview** **3**

1.1 Simulator . . . 3

**2 From T-DV1 to UVM** **4**

**3 Environment Architecture** **5**

3.1 One Agent, Three Uses . . . 5

**4 The `mem_if` Agent** **6**

4.1 Widths Versus Geometry . . . 6

4.2 Sequence Item . . . 6

4.3 Driver . . . 7

4.4 Monitor . . . 7

4.5 Agent and Agent Configuration . . . 7

**5 Configuration: `uvm_config_db`** **8**

**6 Scoreboard and Coverage** **9**
6.1 Scoreboard . . . 9
6.2 Coverage Collector . . . 9

**7 Sequences** **10**
7.1 CPU Sequences . . . 10
7.2 Virtual Sequence . . . 11

**8 Tests, Factory and Phases** **11**
8.1 Tests . . . 11
8.2 Type and Instance Overrides . . . 13
8.3 Phases and Objections . . . 13

**9 How Real Projects Organize UVM** **14**

**10 Experiments** **14**

**11 Requirements / Tasks** **15**

**12 Repository and Flow** **15**
12.1 Files and Directories . . . 15
12.2 The `SIM` Variable . . . 16
12.3 Example Commands . . . 16

**13 Deliverables** **17**

**14 Assessment** **17**

**15 Review Questions** **17**

**16 References** **18**

# 1 Overview

In Lab T-DV1 you built a layered testbench by hand: generator, driver, monitors, scoreboard, environment and test, connected with mailboxes, events and method calls. Every team that builds testbenches this way ends up inventing the same pieces again, slightly differently each time. UVM is the industry standard that fixes those pieces: how components are created, configured, connected, started and stopped, and how stimulus is written. Almost every verification job uses it, and the large open-source RISC-V projects (OpenTitan, CVA6) verify their blocks with it.

Because you built each piece yourself first, this lab is mostly a translation. For every UVM concept, ask: *which part of my T-DV1 testbench does this replace, and what does it give me that mine did not?*

**Where this lab sits.** The design under test is the `dcache` designed in Lab R4 and verified in Lab V3, connected to the `arb_mem` of Labs R3/V2 exactly as in the V3 and T-DV1 testbenches. The reference model is the `cache_model_pkg` of Lab T-MOD, reused without a single change; the scoreboard algorithm and the injected bugs come from Lab T-DV1; the covergroup comes from Lab V3. This is the last lab of the DV track. Its outputs — the reusable `mem_if` agent in `dv/uvm/mem_if_agent/` and the `SIM` selection in the flow — are the starting point of every later UVM environment in the repository.

**By the end of this lab you should be able to:**

1. Map every part of a hand-built layered testbench to its UVM equivalent.
2. Write a reusable agent (sequence item, sequencer, driver, monitor, configuration) for one interface protocol, and instantiate it as active or passive.
3. Write sequences and a virtual sequence, and select stimulus with factory overrides instead of code changes.
4. Pass virtual interfaces and configuration objects through `uvm_config_db`.
5. Explain the UVM phases and end a test correctly with objections.
6. Connect monitors, a scoreboard and a coverage collector through analysis ports.
7. Organize a UVM environment in the OpenTitan-style layout (`env/`, `seq_lib/`, `tests/`, `tb/`) and run it from the common make flow with a second simulator.

## 1.1 Simulator

UVM is a large SystemVerilog class library and needs a simulator with complete support for it. Open-source support for UVM is still maturing (recent Verilator 5.x releases compile parts of UVM, but not reliably enough for this lab); the coordinator decides which simulator the team uses (the team's commercial simulator, or EDA Playground for practice). The flow already selects the simulator with the `SIM` make variable and one rules file per tool, `flow/mk/sim_<tool>.mk` (Lab L0); this lab adds the switch `UVM=1`, which compiles the UVM library into the model and passes `TEST` as `+UVM_TESTNAME` (Section 12). The seed option differs between tools (for example `-sv_seed`, `-svseed` or `+ntb_random_seed=`); the rules file hides it behind the existing `SEED` variable.

**On the Synopsys cloud:** VCS has UVM 1.2 built in: `vcs -ntb_opts uvm-1.2` compiles it, nothing is added to the file list. In the flow: `make sim SIM=vcs UVM=1` (for the dcache, `DV_ENV=dv_uvm` implies `UVM=1`); `TEST=dcache_smoke_test` becomes `+UVM_TESTNAME=dcache_smoke_test`, the seed becomes `+ntb_random_seed=`. Verbosity: `PLUSARGS=+UVM_VERBOSITY=UVM_HIGH`. Debug in Verdi: `WAVES=1` builds with `-kdb -debug_access+all` and dumps an FSDB (`make waves`); add `PLUSARGS="+UVM_VERDI_TRACE=UVM_AWARE +UVM_TR_RECORD"` to record the UVM hierarchy and the sequence items, so that Verdi shows transactions next to the waves. Verilator 5.013 (the cloud version) cannot run UVM.

## 2 From T-DV1 to UVM

| **T-DV1** | **UVM** | **What UVM adds** |
|---|---|---|
| `cpu_txn` | `uvm_sequence_item` | Standard `copy`, `compare`, `print`; factory creation |
| generator + blueprint | `uvm_sequence` + `uvm_sequencer` | Stimulus as reusable, layerable sequences; arbitration between sequences |
| mailbox gen → drv | `seq_item_port` (`get_next_item`/`item_done`) | Built-in handshake: the sequence knows when the driver finished |
| driver, monitors (`dv/vip/mem_if/`) | `uvm_driver`, `uvm_monitor` | Standard structure; grouped into an `uvm_agent` |
| monitor → `scb.write()` | `uvm_analysis_port` | One port, any number of subscribers, no direct handles |
| `cache_env` | `uvm_env` | Phased construction and connection |
| test class + `case` on `+TEST` | `uvm_test` + `+UVM_TESTNAME` | Test selected by name through the factory |
| blueprint replacement | factory override | Replace any class anywhere without editing the code that creates it |
| handles passed by hand | `uvm_config_db` | Hierarchical, type-safe configuration |
| `gen_done` event, watchdog | objections, phase timeout | Standard end-of-test mechanism |
| `$display`, error counters | `` `uvm_info``/`` `uvm_error``, report server | Verbosity control, uniform summary |
| `cache_model` (T-MOD) | `cache_model` (T-MOD) | Nothing: the model is reused unchanged |

The last row matters: the reference model is plain SystemVerilog, not UVM. Keeping models free of the methodology keeps them reusable in any environment, including the T-DV1 testbench and the model-only exploration testbenches of Labs T-AM1 and T-AM2 in `dv/model/explore/`.

# 3 Environment Architecture

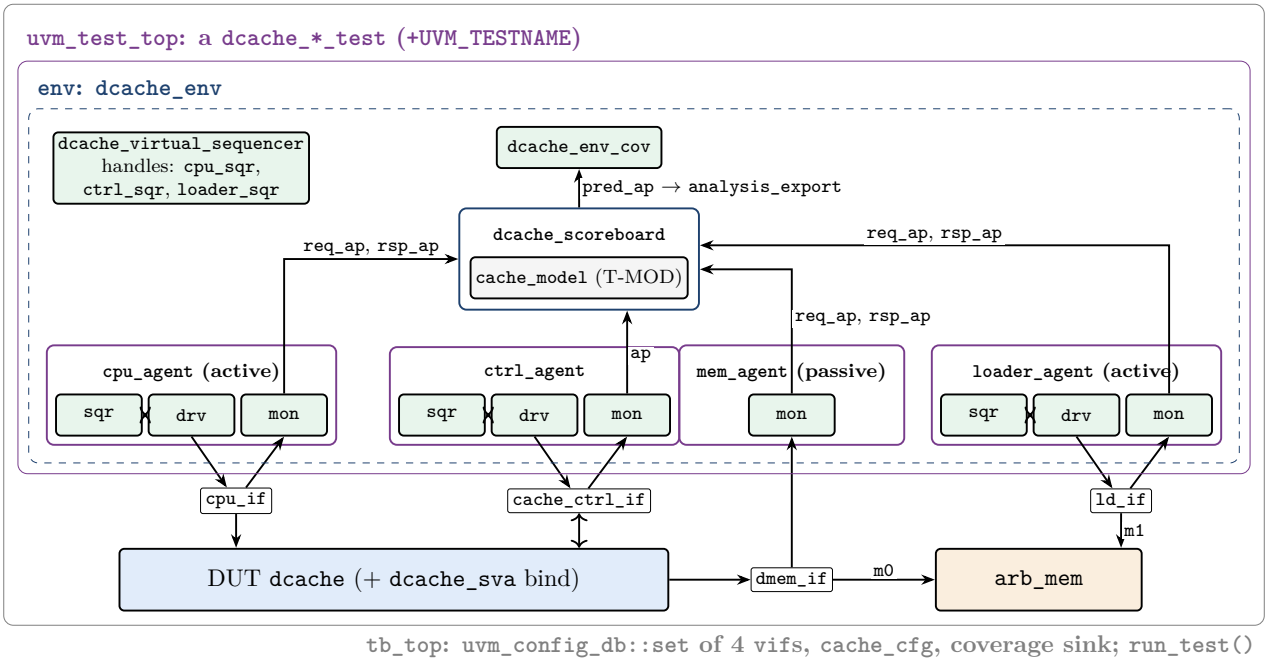



Figure 1: Component hierarchy and TLM connections of the T-DV2 environment. One `mem_if_agent` class is used three times (purple frames, `sqr`/`drv`/`mon` are its sequencer, driver and monitor); arrows into the scoreboard are analysis ports connected to its `uvm_analysis_imp_*` exports in `dcache_env::connect_phase`; double arrows inside an agent are the `seq_item_port`–`seq_item_export` connection. The virtual sequencer only holds handles of the three sequencers.

Figure 1 shows what the environment builds and how it is connected; Figure 2 (Section 7) shows how an item travels from a sequence to the pins, and Figure 3 (Section 8) the phases.

## 3.1 One Agent, Three Uses

The cache system has three `mem_if` interfaces: the CPU side of the cache, its memory side, and the loader port `m1` of `arb_mem`. Write **one** agent for the `mem_if` protocol and instantiate it three times with different configurations:

| Instance | Mode | Role |
|---|---|---|
| `cpu_agent` | active | Drives CPU requests into the cache; its monitor reports accepted requests and completed reads |
| `mem_agent` | passive | Only monitors the cache's memory-side traffic; no driver, no sequencer |
| `loader_agent` | active | Preloads memory before the test and reads it back after the final flush |

The agent belongs in `dv/uvm/mem_if_agent/`, next to the non-UVM classes of Lab T-DV1 in `dv/vip/mem_if/`: it is verification IP for the protocol, not for the cache, and any future block with a `mem_if` port will use it again. A fourth, small agent drives and observes the cache control signals (`flush`, `flush_done`, `perf`) through a new `cache_ctrl_if` interface in `hw/ip/dcache/dv_uvm/tb/`. Because it is specific to the cache, the control agent lives in the cache environment (`hw/ip/dcache/dv_uvm/env/ctrl_agent/`), not in `dv/uvm/`.

# 4 The mem_if Agent

## 4.1 Widths Versus Geometry

Class types must be fixed at compile time, so bit widths remain class parameters (as in Labs T-MOD and T-DV1). The agent classes are parameterized by `AW` and `DW` and registered with `` `uvm_object_param_utils `` / `` `uvm_component_param_utils ``. The tests, which the factory must find by name through `+UVM_TESTNAME`, are not parameterized; they use the widths from a small package, `dcache_tb_cfg_pkg` (`hw/ip/dcache/dv_uvm/tb/`), set by macros from `DEFINES` in the Makefile. Cache geometry (lines, words per line) stays a run-time configuration object, the T-MOD `cache_cfg`, exactly as in Lab T-DV1.

## 4.2 Sequence Item

```
// dv/uvm/mem_if_agent/mem_if_item.sv
class mem_if_item #(int AW = 10, int DW = 32) extends uvm_sequence_item;
  `uvm_object_param_utils(mem_if_item #(AW, DW))

  rand bit          we;
  rand logic [AW-1:0] addr;
  rand logic [DW-1:0] wdata;
  rand int unsigned idle;
  logic      [DW-1:0] rdata;      // filled in by the monitor

  constraint c_idle { idle dist { 0 := 70, [1:3] :/ 30 }; }   // ':/' spreads 30 over
    1..3

  function new(string name = "mem_if_item");
    super.new(name);
  endfunction

  virtual function void do_copy(uvm_object rhs);
    mem_if_item #(AW, DW) r;
    if (!$cast(r, rhs)) `uvm_fatal("CAST", "do_copy: wrong type")
    super.do_copy(rhs);
    we = r.we; addr = r.addr; wdata = r.wdata; idle = r.idle; rdata = r.rdata;
  endfunction

  virtual function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    mem_if_item #(AW, DW) r;
    if (!$cast(r, rhs)) return 0;
    return super.do_compare(rhs, comparer) &&
           we == r.we && addr == r.addr && (!we || wdata == r.wdata);
  endfunction

  virtual function string convert2string();
    return $sformatf("%s addr=%h wdata=%h rdata=%h",
                     we ? "WR" : "RD", addr, wdata, rdata);
  endfunction
endclass
```

Implement `do_copy`, `do_compare` and `convert2string` by hand rather than with the field automation macros (`` `uvm_field_int ``). Explain in the README what the macros would do and why many teams (OpenTitan among them) avoid them.

### 4.3 Driver

```
// dv/uvm/mem_if_agent/mem_if_driver.sv
class mem_if_driver #(int AW = 10, int DW = 32)
  extends uvm_driver #(mem_if_item #(AW, DW));
  `uvm_component_param_utils(mem_if_driver #(AW, DW))

  virtual mem_if #(AW, DW) vif;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(virtual mem_if #(AW, DW))::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "virtual interface not set")
  endfunction

  virtual task run_phase(uvm_phase phase);
    vif.req_valid <= 1'b0;
    @(posedge vif.rst_n);
    forever begin
      seq_item_port.get_next_item(req);
      drive(req);                  // T-DV1 driver logic, now a method
      seq_item_port.item_done();
    end
  endtask
endclass
```

The pin-level code of `drive()` is the same as in the T-DV1 driver class (which itself came from the V2 driver task). Either copy it, or — better — let the UVM driver own an object of the T-DV1 driver class from `dv/vip/mem_if/` and call it. Say in the README which option you chose and why.

### 4.4 Monitor

The monitor has two analysis ports, because subscribers need two different moments:

- `req_ap`: every accepted request, at the acceptance edge. The scoreboard uses it to run the model at the moment the cache accepts a CPU request (the T-DV1 algorithm), and to collect memory-side traffic.
- `rsp_ap`: every completed read, with `rdata` filled in, one cycle later.

When a read response and a new acceptance happen at the same edge, publish the response first. Explain why the order matters for the scoreboard.

### 4.5 Agent and Agent Configuration

```
// dv/uvm/mem_if_agent/mem_if_agent_cfg.sv
class mem_if_agent_cfg extends uvm_object;
  `uvm_object_utils(mem_if_agent_cfg)
  uvm_active_passive_enum is_active = UVM_ACTIVE;
  function new(string name = "mem_if_agent_cfg"); super.new(name); endfunction
endclass

// In mem_if_agent::build_phase:
```

```
//   monitor is always created;
//   sequencer and driver only if cfg.is_active == UVM_ACTIVE.
// In mem_if_agent::connect_phase (active only):
//   driver.seq_item_port.connect(sequencer.seq_item_export);
```

All agent files are `` `included `` by one package, `mem_if_agent_pkg`, which is the only entry of the agent's file list `mem_if_agent.f` besides its `+incdir+`. The agent depends only on `uvm_pkg` and the `mem_if` interface of Lab R3 — never on anything in `hw/ip/dcache/`.

## 5 Configuration: `uvm_config_db`

The top module places the virtual interfaces and the cache configuration into the configuration database before the test starts. The test builds the environment configuration object `dcache_env_cfg` (one `mem_if_agent_cfg` per agent plus the T-MOD `cache_cfg`) and passes it down; components retrieve what they need in `build_phase`.

```
// hw/ip/dcache/dv_uvm/tb/tb_top.sv
module tb_top;
  import uvm_pkg::*;
  import dcache_tb_cfg_pkg::*;
  import cache_model_pkg::*;     // T-MOD, unchanged
  // clock, reset, interfaces, cache_ctrl_if, dcache, arb_mem (as in T-DV1)

  initial begin
    cache_cfg cfg = new();
    cfg.num_lines  = NUM_LINES;      // tb_top parameters, set by -G (PARAMS)
    cfg.line_words = LINE_WORDS;
    uvm_config_db #(cache_cfg)::set(null, "uvm_test_top", "cache_cfg", cfg);

    uvm_config_db #(virtual mem_if #(ADDR_W, DW))::set(
        null, "uvm_test_top.env.cpu_agent*",    "vif", cpu_if);
    uvm_config_db #(virtual mem_if #(ADDR_W, DW))::set(
        null, "uvm_test_top.env.mem_agent*",    "vif", mem_side_if);
    uvm_config_db #(virtual mem_if #(ADDR_W, DW))::set(
        null, "uvm_test_top.env.loader_agent*", "vif", loader_if);
    uvm_config_db #(virtual cache_ctrl_if)::set(
        null, "uvm_test_top.env.ctrl_agent*",   "vif", ctrl_if);

    run_test();      // test chosen by +UVM_TESTNAME
  end
endmodule
```

The most common UVM error is a `get()` that fails because the *type* (including parameters) or the *path* does not match the `set()` exactly. When you hit it, use `+UVM_CONFIG_DB_TRACE` and record in the README what was wrong.

The repository conventions still hold: `tb_top` prints the configuration (PARAMS, test name, seed, plusargs) at the start, has a watchdog — here the UVM phase timeout, set with `+UVM_TIMEOUT` or `uvm_top.set_timeout()` — and the simulation ends with exactly one line `TEST PASSED` or `TEST FAILED` (Section 8).

# 6 Scoreboard and Coverage

## 6.1 Scoreboard

The scoreboard receives several streams. One analysis *port* can feed many subscribers, but a component with several inputs needs one analysis *export* per stream, each with its own `write` method:

```
// hw/ip/dcache/dv_uvm/env/dcache_scoreboard.sv
`uvm_analysis_imp_decl(_cpu_req)
`uvm_analysis_imp_decl(_cpu_rsp)
`uvm_analysis_imp_decl(_mem_req)
`uvm_analysis_imp_decl(_mem_rsp)
`uvm_analysis_imp_decl(_ld_req)
`uvm_analysis_imp_decl(_ld_rsp)
`uvm_analysis_imp_decl(_ctrl)

class dcache_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(dcache_scoreboard)
  uvm_analysis_imp_cpu_req #(item_t, dcache_scoreboard) cpu_req_export;
  uvm_analysis_imp_cpu_rsp #(item_t, dcache_scoreboard) cpu_rsp_export;
  uvm_analysis_imp_mem_req #(item_t, dcache_scoreboard) mem_req_export;
  // ... mem_rsp, ld_req, ld_rsp and ctrl exports

  cache_model_t model;   // T-MOD model, unchanged
  uvm_analysis_port #(dcache_cov_txn) pred_ap; // compared predictions, for coverage

  virtual function void write_cpu_req(item_t t); /* run model, compare mem */
    endfunction
  virtual function void write_cpu_rsp(item_t t); /* compare read data      */
    endfunction
  virtual function void write_mem_req(item_t t); /* collect observed mem   */
    endfunction
  // ...
endclass
```

The comparison algorithm is the one from Lab T-DV1 (memory traffic collected and compared in order when the CPU request is accepted, read data from a queue of expectations, flush write-backs compared as a set). In addition:

- `check_phase`: no expected read data and no observed memory traffic may be left over; the DUT performance counters reported at the final `flush_done` must equal the model statistics.
- The loader readback after the final flush is compared with the model's backing memory.
- Every mismatch is a `` `uvm_error `` with both values in the message.

## 6.2 Coverage Collector

Reuse the Lab V3 covergroup (`cache_cg`, with its coverage plan: operation × result, index, offset on miss, back-to-back, flush) in `dcache_env_cov`, an `uvm_subscriber` connected to the scoreboard's `pred_ap`. Keep the coverpoints and bins identical to V3, so the coverage numbers of both environments can be compared.

**Test term — Coverage closure.** The completion criterion of the regression: every bin of the Lab V3 coverage plan hit, merged over all runs, in every `PARAMS` set. Identical bins in all three environments make the numbers comparable: the same plan closed by the same tests shows that the UVM stimulus is as good as the hand-built one.

The coverage collector therefore samples *predicted* results. Explain why sampling predictions is acceptable here (hint: every prediction is compared with the DUT) and when it would not be.

# 7 Sequences

All sequences live in `hw/ip/dcache/dv_uvm/seq_lib/` and are `` `included `` by `dcache_seq_pkg` (or directly by the env package — choose one and keep it consistent).

## 7.1 CPU Sequences

```
class cpu_random_seq extends uvm_sequence #(item_t);
  `uvm_object_utils(cpu_random_seq)
  int unsigned num_txn = 1000;

  function new(string name = "cpu_random_seq"); super.new(name); endfunction

  virtual task body();
    repeat (num_txn) begin
      req = item_t::type_id::create("req");   // factory create
      start_item(req);
      if (!req.randomize()) `uvm_error("RAND", "randomize failed")
      finish_item(req);
    end
  endtask
endclass
```

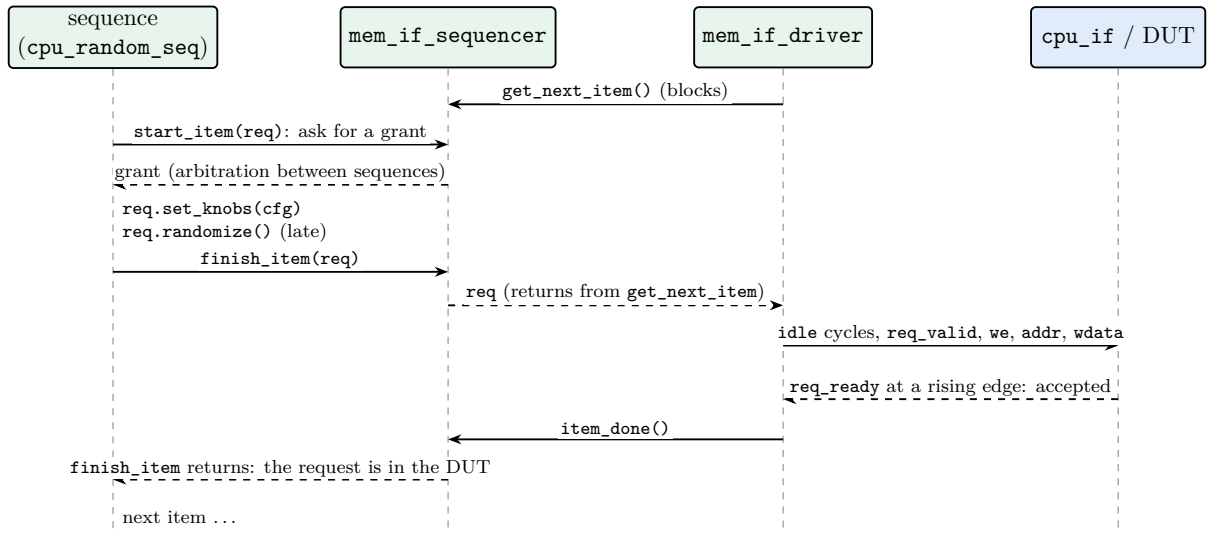



Figure 2: Sequence–sequencer–driver handshake for one item. The sequence blocks in `start_item` until the sequencer grants it, randomizes the item as late as possible, and blocks in `finish_item` until the driver calls `item_done()` — in this agent, when the request has been *accepted* by the DUT. The T-DV1 mailbox only told the generator that the driver had *taken* the item.

Required CPU sequences: `cpu_random_seq`, `cpu_sequential_seq`, `cpu_fits_seq` and `cpu_directed_seq` (a fixed list of transactions for the smoke test). Loader sequences: `mem_preload_seq` and `mem_readback_seq`. Control sequence: `flush_seq`. The number of transactions comes from the `+NUM_TXN=` plusarg (`PLUSARGS` in make), as in V3 and T-DV1.

Because `req` is created through the factory, a test can make `cpu_random_seq` produce conflict transactions without changing the sequence — the UVM version of the T-DV1 blueprint pattern.

### 7.2 Virtual Sequence

One test step involves three agents: preload memory, run CPU traffic with optional flushes, flush, read back. A **virtual sequence** runs on a virtual sequencer (`dcache_virtual_sequencer`, in `env/`) that holds handles to the agents' sequencers and starts sub-sequences on them:

```
// hw/ip/dcache/dv_uvm/seq_lib/dcache_main_vseq.sv
class dcache_main_vseq extends uvm_sequence;
  `uvm_object_utils(dcache_main_vseq)
  `uvm_declare_p_sequencer(dcache_virtual_sequencer)

  virtual task body();
    mem_preload_seq  pre  = mem_preload_seq::type_id::create("pre");
    cpu_random_seq   cpu  = cpu_random_seq::type_id::create("cpu");
    flush_seq        fl   = flush_seq::type_id::create("fl");
    mem_readback_seq rb   = mem_readback_seq::type_id::create("rb");

    pre.start(p_sequencer.loader_sqr);
    cpu.start(p_sequencer.cpu_sqr);   // with +FLUSH_EVERY=N: N items, flush, N items,
    ...
    fl.start(p_sequencer.ctrl_sqr);   // final flush
    rb.start(p_sequencer.loader_sqr);
  endtask
endclass
```

A flush may only be pulsed while the CPU side is idle. Running the CPU traffic and the flushes in parallel (`fork`) would need a handshake between the two; it is simpler to run the CPU sequence in chunks of `FLUSH_EVERY` items and start `flush_seq` between two chunks: when a CPU sequence has finished, its last `finish_item()` has returned, i.e. the last request has been accepted (Figure 2). The stream is the same as in Lab V3 (a flush before every `FLUSH_EVERY`-th access).

## 8 Tests, Factory and Phases

### 8.1 Tests

```
// hw/ip/dcache/dv_uvm/tests/dcache_base_test.sv
class dcache_base_test extends uvm_test;
  `uvm_component_utils(dcache_base_test)
  dcache_env env;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // create dcache_env_cfg, fill it, uvm_config_db::set for "env*"
    env = dcache_env::type_id::create("env", this);
  endfunction

  virtual function void end_of_elaboration_phase(uvm_phase phase);
    uvm_top.print_topology();
  endfunction

  virtual task run_phase(uvm_phase phase);
    dcache_main_vseq vseq = dcache_main_vseq::type_id::create("vseq");
    phase.raise_objection(this);
```

```
    vseq.start(env.vsqr);
    phase.drop_objection(this);
  endtask

  virtual function void report_phase(uvm_phase phase);
    uvm_report_server svr = uvm_report_server::get_server();
    if (svr.get_severity_count(UVM_ERROR) + svr.get_severity_count(UVM_FATAL) == 0)
      $display("TEST PASSED");
    else
      $display("TEST FAILED");
  endfunction
endclass

class dcache_conflict_test extends dcache_base_test;
  `uvm_component_utils(dcache_conflict_test)
  virtual function void build_phase(uvm_phase phase);
    dcache_item::type_id::set_type_override(conflict_item::get_type());
    super.build_phase(phase);
  endfunction
endclass
```

The pass/fail line is printed with a plain `$display`, so that the log contains exactly one line `TEST PASSED` or `TEST FAILED` and the existing check of the `sim` target works unchanged.

Required tests: `dcache_smoke_test`, `dcache_random_test`, `dcache_sequential_test`, `dcache_fits_test`, `dcache_conflict_test` and `dcache_flush_test` (the V3 and T-DV1 test set). Each test must differ from the base test only through configuration and factory overrides — never by copying environment code.

**Test term — Smoke test.** The shortest test that exercises every basic behavior once (here the ten directed accesses of Lab R4: hit, clean and dirty miss, write miss) through the whole UVM environment. It shows in seconds that the factory, the configuration database, the agents, the model and the RTL work together; a failure means nothing else is worth running. It stays the first test of the regression.

**Test term — Directed and constrained-random tests.** A directed test sends a fixed list of accesses with a known outcome (`smoke`, `sequential`, the directed parts of `flush`); a constrained-random test draws them with `randomize()` under constraints (`fits`, `conflict`, `random`), with a printed seed for reproduction. Directed tests are easy to debug; constrained-random tests, checked by the model, find what nobody thought of.

| Test | How it differs from the base test — what it shows |
|---|---|
| `dcache_smoke_test` | `cpu_random_seq` → `cpu_directed_seq`: the R4 cases |
| `dcache_sequential_test` | → `cpu_sequential_seq`: stride-1 reads over 4× the capacity, one miss per line |
| `dcache_fits_test` | → `cpu_fits_seq`: working set inside the capacity, only cold misses |
| `dcache_random_test` | base stimulus with `+FLUSH_EVERY`: everything mixed, periodic flushes |
| `dcache_conflict_test` | `dcache_item` → `conflict_item` (type override): one index, conflict misses, dirty evictions |
| `dcache_flush_test` | `dcache_main_vseq` → `dcache_flush_vseq`: flush with 0, 1, some, all lines dirty |
| `dcache_backpressure_test` | loader driver → a delaying driver (instance override): the loader port's handshake with stalls, the CPU side unchanged |

**Test term — Regression.** All tests × seeds × `PARAMS` sets of `dv_uvm/regress.list`, run by one command (`make regress DV_ENV=dv_uvm`) with a pass/fail summary and the merged coverage. Run before every merge; it is the evidence that every test passes for more than the one seed it was debugged with.

### 8.2 Type and Instance Overrides

- A **type override** replaces a class everywhere it is created through the factory (`dcache_conflict_test`).
- An **instance override** replaces it only below one path in the hierarchy. Write `dcache_backpressure_test` (*back-pressure test*: see the box below), which replaces the driver of `loader_agent` only — not of `cpu_agent` — with a derived driver that inserts random idle cycles, using `set_inst_override_by_type`.
- Print the factory configuration (`factory.print()`) in both tests and include the output in the README.

**Test term — Back-pressure test.** One side of a handshake is slowed down at random so that every stall path is exercised: here the loader driver inserts random idle cycles before its requests, so preload and read-back run with gaps. A pass shows that nothing in the environment (or the DUT) silently assumes back-to-back traffic; doing it with an instance override shows that the rest of the environment did not change.

### 8.3 Phases and Objections

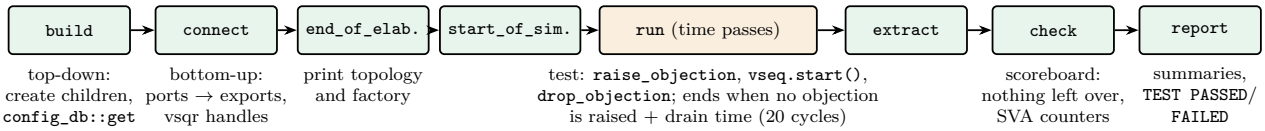



Figure 3: UVM phases used in this lab. Only `run` consumes time; the function phases run in zero time, `build` from the test down to the agents, `connect` from the leaves up.

| Phase | Use in this lab |
|---|---|
| `build_phase` | Create children through the factory; get configuration (top-down) |
| `connect_phase` | Connect driver to sequencer, analysis ports to exports, sequencer handles into the virtual sequencer (bottom-up) |
| `end_of_elaboration_phase` | Print the topology |
| `run_phase` | The only time-consuming phase used here; the test raises and drops an objection |
| `check_phase` | Scoreboard end-of-test checks |
| `report_phase` | Summaries; `TEST PASSED` or `TEST FAILED` |

The run phase ends when no objection is raised. Answer in the README: what happens if the test forgets `raise_objection`? What happens if it drops the objection immediately after the last CPU transaction has been *sent*, while the cache is still handling it? Use a drain time or an explicit wait, and justify your choice.

## 9 How Real Projects Organize UVM

The layout of this lab is a simplified copy of OpenTitan. Before you start, spend an hour reading one real environment; it will answer many "where does this go?" questions.

- **OpenTitan, shared library** (`hw/dv/sv/`): `dv_lib` provides base classes (`dv_base_env`, `dv_base_env_cfg`, `dv_base_scoreboard`, `dv_base_vseq`, `dv_base_test`) that every IP environment extends; protocol agents such as `tl_agent` (TileLink-UL) sit next to it — the equivalent of `dv/uvm/mem_if_agent/`.
- **OpenTitan, per IP** (`hw/ip/<ip>/dv/`): `env/` (`<ip>_env`, `<ip>_env_cfg`, `<ip>_scoreboard`, `<ip>_env_cov`, `<ip>_virtual_sequencer`, and `seq_lib/` with `<ip>_base_vseq` and one vseq per test), `tests/`, `tb/tb.sv`, plus a testplan (`data/<ip>_testplan.hjson`) and a simulation configuration (`<ip>_sim_cfg.hjson`) that maps test names to vseqs for the regression tool `dvsim`.
- **CVA6** (`verif/env/uvme/`): the core-level UVM environment (`uvme_cva6_env`, configuration, coverage, virtual sequences) built on reusable agents in `verif/env/` and `uvma_*` agents from the OpenHW core-v-verif library, with a reference-model-based scoreboard.

Note what both projects share with your environment: protocol agents are separated from block environments, configuration objects are passed down from the test, and every test is a thin class that selects a virtual sequence and some overrides.

## 10 Experiments

**Test term — Negative / error-injection test.** A known bug is put into the RTL on purpose and the regression tests are run again: they must fail, and the first error must name the right check. It proves that the UVM scoreboard, the analysis connections and the assertions really fire — a scoreboard whose export is not connected passes everything — and comparing the detection point with Lab T-DV1 shows that the translation lost nothing.

1. **Same bugs, new testbench.** Rerun the six injected bugs of Lab T-DV1 (one at a time, in a copy of the RTL, never committed on the lab branch). Every bug must be found; record the detection point and compare it with T-DV1:

| Injected bug (T-DV1) | Detected by | After txn # | T-DV1 |
|---|---|---|---|
| Write-back uses the new tag instead of the victim's tag | | | |
| Dirty bit not cleared after refill | | | |
| Refill writes word 1 into word 0 | | | |
| Write hit does not set the dirty bit | | | |
| `misses` counter also counts hits | | | |
| Flush skips the last line | | | |

2. **Reuse check.** Without editing the agent, add a fourth `mem_if_agent` instance in passive mode on the loader port and connect it to a simple protocol checker that counts transactions. Measure how many lines of code this took.
3. **Verbosity.** Run one test with `+UVM_VERBOSITY=UVM_LOW` and `UVM_HIGH`. Choose the verbosity of every `` `uvm_info `` in your code so that `UVM_LOW` gives a readable one-screen summary and `UVM_HIGH` is enough to debug a mismatch.
4. **Comparison.** Compare the T-DV1 and T-DV2 testbenches: lines of code, time to add a new test, time to add a new interface. Where did UVM cost more effort, and where did it pay off?

# 11 Requirements / Tasks

1. Add `UVM=1` to the `flow/mk/sim_<tool>.mk` of the chosen simulator (UVM library, `TEST` as `+UVM_TESTNAME`, own model directory), keeping its targets (`build`, `sim`, `waves`, `cov`) and variables (`TEST`, `SEED`, `PLUSARGS`, `PARAMS`, `DEFINES`, `WAVES`); with `UVM=0` nothing changes and `make check-all` still passes for every IP.
2. Write the `mem_if` agent in `dv/uvm/mem_if_agent/`: item, agent configuration, sequencer, driver, monitor with `req_ap` and `rsp_ap`, agent; active and passive modes; no dependency on the cache.
3. Write `cache_ctrl_if` and the control agent (flush request, `flush_done` and `perf` observation).
4. Write `tb_top`, `dcache_tb_cfg_pkg` and the `uvm_config_db` set-up for all four virtual interfaces and the `cache_cfg`.
5. Write `dcache_env`, `dcache_env_cfg`, `dcache_virtual_sequencer`, `dcache_scoreboard` (the T-DV1 algorithm, the T-MOD model unchanged, `check_phase` checks) and `dcache_env_cov` (the V3 covergroup).
6. Write the sequences, the virtual sequence and the seven tests (six functional tests plus `dcache_backpressure_test`), using a type override in `dcache_conflict_test` and an instance override in `dcache_backpressure_test`.
7. End every test cleanly with objections (no `$finish` in the environment), print exactly one `TEST PASSED` / `TEST FAILED` line, and set a phase timeout.
8. Write `hw/ip/dcache/dv_uvm/regress.list` and run the regression: every test with at least 5 seeds, for the default geometry and at least one other `PARAMS` set; all pass and the V3 coverage plan is closed.
9. Perform the four experiments of Section 10.

# 12 Repository and Flow

## 12.1 Files and Directories

```
flow/mk/common.mk                SIM ?= verilator; includes sim_$(SIM).mk   (L0)
flow/mk/sim_<tool>.mk            UVM=1: UVM library, TEST -> +UVM_TESTNAME   (changed)
dv/uvm/mem_if_agent/             reusable UVM agent for mem_if                (new)
+-- mem_if_agent_pkg.sv          `includes the files below
+-- mem_if_item.sv, mem_if_agent_cfg.sv, mem_if_sequencer.sv
+-- mem_if_driver.sv, mem_if_monitor.sv, mem_if_agent.sv
+-- mem_if_agent.f, README.md
dv/model/cache_model/                T-MOD cache_model_pkg                        (
    unchanged)
hw/ip/dcache/dv_uvm/                                                          (new)
+-- env/                         dcache_env_pkg.sv, dcache_env.sv, dcache_env_cfg.sv,
|                                dcache_scoreboard.sv, dcache_env_cov.sv,
|                                dcache_virtual_sequencer.sv, ctrl_agent/
+-- seq_lib/                     cpu_*_seq.sv, mem_*_seq.sv, flush_seq.sv,
|                                conflict_item.sv, dcache_main_vseq.sv, ...
+-- tests/                       dcache_test_pkg.sv, dcache_base_test.sv, dcache_*_test.
    sv
+-- tb/                          tb_top.sv, dcache_tb_cfg_pkg.sv
|                                (cache_ctrl_if.sv reused from dv_layered/tb/)
+-- dv_uvm.f                     uvm agent, model, env, seq_lib, tests, tb (nested -f)
+-- regress.list                 tests x seeds x PARAMS
hw/ip/dcache/Makefile            DV_ENV selection, see below                  (changed)
```

```
hw/ip/dcache/README.md           new section "UVM environment"              (changed)
hw/ip/dcache/doc/verification.md T-DV2 results                              (changed)
```

`dv_uvm.f` follows the file-list rules of Lab L0: packages before users, all paths start with `${REPO_ROOT}`, nested `-f` for the RTL (`hw/ip/dcache/rtl/dcache.f`, `hw/ip/arb_mem/rtl/arb_mem.f`), the agent (`dv/uvm/mem_if_agent/mem_if_agent.f`) and the model in `dv/model/cache_model/`. The UVM library itself is *not* in the file list: each `sim_<tool>.mk` adds it in the tool's own way (most commercial simulators have a `-uvm` switch or a precompiled library).

## 12.2 The `SIM` Variable

```
# flow/mk/common.mk (excerpt)
SIM ?= verilator
include $(REPO_ROOT)/flow/mk/sim_$(SIM).mk

# flow/mk/sim_<tool>.mk (sketch; the coordinator provides the tool options)
SIM_SEED_OPT  = <tool seed option>$(SEED)      # e.g. -sv_seed / -svseed
build: ...    # compile $(TB_FLIST), -G/-D from PARAMS/DEFINES; UVM=1: + UVM library
sim:   build  # run $(SIM_SEED_OPT) $(PLUSARGS) (UVM=1: +UVM_TESTNAME=$(TEST)),
              # log to build/<ip>/<cfg>/sim/$(TEST)_s$(SEED).log,
              # fail unless the log has TEST PASSED and no TEST FAILED
```

With the default `UVM=0` nothing changes for earlier labs. A UVM model is a different build, so it goes into its own subdirectory (`build/<ip>/<cfg>/sim/<tool>_uvm/`): two variants never share compiled output. For UVM environments `TEST` names the UVM test class and is passed as `+UVM_TESTNAME`; the test's `report_phase` prints `TEST PASSED` only with no `UVM_ERROR`/`UVM_FATAL`, so the existing check of the `sim` target needs no change.

**On the Synopsys cloud:** On the cloud `flow/mk/sim_vcs.mk` implements this: `UVM=1` adds `-ntb_opts uvm-1.2` to the `vcs` command and `+UVM_TESTNAME=$(TEST)` to the `simv` command, next to `+ntb_random_seed=$(SEED)`; the model goes to `sim/vcs_uvm/` (`vcs_dbg_uvm/` with `WAVES=1`, `vcs_cov_uvm/` with `COV=1`). Coverage: `make regress DV_ENV=dv_uvm COV=1`, then `make cov DV_ENV=dv_uvm PARAMS="..."` (URG; the reused covergroups appear under "Groups").

The `dcache` now has three verification environments (`dv/`, `dv_layered/`, `dv_uvm/`). The IP Makefile selects one with an IP-local variable:

```
# hw/ip/dcache/Makefile (excerpt)
DV_ENV ?= dv                          # dv | dv_layered | dv_uvm
ifeq ($(DV_ENV),dv_uvm)
  TB_FLIST     := dv_uvm/dv_uvm.f
  TB_TOP       := tb_top
  REGRESS_LIST := dv_uvm/regress.list
  override UVM := 1                   # this environment needs the UVM library
endif
```

## 12.3 Example Commands

```
source env.sh
make sim IP=dcache SIM=<tool> DV_ENV=dv_uvm TEST=dcache_smoke_test
make sim IP=dcache SIM=<tool> DV_ENV=dv_uvm TEST=dcache_random_test \
     SEED=42 PLUSARGS="+NUM_TXN=5000 +UVM_VERBOSITY=UVM_LOW"
make sim IP=dcache SIM=<tool> DV_ENV=dv_uvm TEST=dcache_conflict_test \
     PARAMS="NUM_LINES=8 LINE_WORDS=8"
make sim IP=dcache SIM=<tool> DV_ENV=dv_uvm TEST=dcache_smoke_test \
```

```
    PLUSARGS=+UVM_CONFIG_DB_TRACE        # debug a failing config_db get()
make regress IP=dcache SIM=<tool> DV_ENV=dv_uvm
make -C $REPO_ROOT check-all             # earlier labs still pass (Verilator)
```

Work on branch `lab/tdv2-uvm`; tag `tdv2-done` when the merge request is accepted.

## 13 Deliverables

The merge request contains all files of Section 12, a passing regression and the following documentation.

`dv/uvm/mem_if_agent/README.md`: what the agent does, its configuration knobs, active/passive use, the two analysis ports and their ordering rule, and a minimal instantiation example.

`hw/ip/dcache/README.md`, section "UVM environment", and `hw/ip/dcache/doc/verification.md` must contain:

- The T-DV1 to UVM mapping, in your own words, with one sentence on what each UVM feature gave you.
- The printed topology and factory configurations.
- The `uvm_config_db` entries: who sets what, for which path, and who gets it.
- The end-of-test mechanism and the answers on objections.
- The field-macro discussion, the driver-reuse choice and the monitor ordering explanation.
- The regression summary, the coverage report (compared with V3) and the experiment results, including the completed bug table.

## 14 Assessment

| Criterion | Weight |
|---|---|
| `mem_if` agent: correctness, active/passive, reuse in three places | 25% |
| Sequences, virtual sequence and factory use | 20% |
| Scoreboard and coverage through analysis ports; model reused unchanged | 20% |
| Configuration, phases and clean end of test | 15% |
| Tests, regression and flow integration (`SIM`, earlier labs unbroken) | 10% |
| Experiments and documentation | 10% |
| Total | 100% |

## 15 Review Questions

1. What does the sequencer–driver handshake (`get_next_item`/`item_done`) give you that the T-DV1 mailbox did not?
2. Why must objects be created with `type_id::create()` rather than `new()` for the factory to work?
3. What is the difference between a type override and an instance override? Give a use for each from your testbench.

4. A `uvm_config_db::get` fails. What are the possible causes, and how do you find which one it is?
5. Why does the monitor have two analysis ports?
6. Why is the `mem_agent` passive, and what would happen if it were active?
7. Why is `build_phase` executed top-down and `connect_phase` bottom-up?
8. What ends the run phase, and what happens if you drop the objection too early?
9. Why is the cache model not a UVM component?
10. Which parts of your environment could another block with a `mem_if` port reuse unchanged, and why does the agent live in `dv/uvm/` and not in `hw/ip/dcache/`?

## 16 References

- Accellera, *Universal Verification Methodology (UVM) User's Guide* and the IEEE 1800.2 UVM standard.
- Verification Academy: UVM Cookbook and UVM basics courses.
- Udemy, *Design Verification with SystemVerilog/UVM*: the UVM sections (see the training program's verification resources).
- Chris Spear and Greg Tumbush, *SystemVerilog for Verification*: chapters on advanced OOP, as background for the factory and callbacks.
- OpenTitan: DV methodology documentation, `hw/dv/sv/` (`dv_lib`, `tl_agent`) and any `hw/ip/<ip>/dv/` environment, https://github.com/lowRISC/opentitan.
- CVA6: `verif/env/uvme/` and the OpenHW core-v-verif library, https://github.com/openhwgroup/cva6.